

02 · pi-ai、pi-agent-core、pi-coding-agent、pi-tui 的职责与依赖

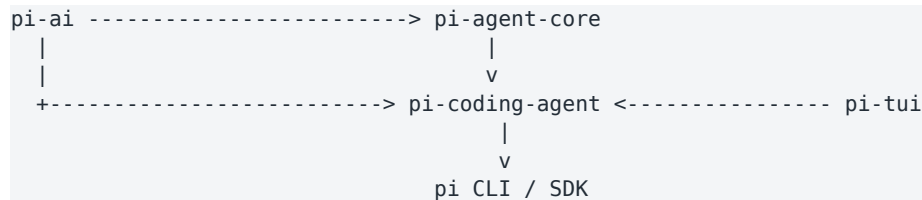
理解四层边界：模型访问、Agent 循环、终端 UI 与最终产品编排

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

四个包不是四个平级功能模块。pi-ai 与 pi-tui 是两个独立底座；pi-agent-core 只依赖 pi-ai；pi-coding-agent 位于最上层，直接组合 pi-ai、pi-agent-core 和 pi-tui。最值得记住的边界是：pi-ai 不执行 Agent loop，pi-agent-core 不理解具体 Coding 产品 UI，pi-tui 不理解 LLM。

1. 依赖图



从 npm 依赖看，pi-agent-core 建立在 pi-ai 之上；pi-coding-agent 同时直接依赖三者。直接依赖 pi-ai 是合理的，因为 Coding Agent 还要管理 Models、Provider 认证、模型选择和摘要调用，而这些并不都通过 Agent 对象间接完成。

2. pi-ai : Provider 无关的 LLM 协议层

pi-ai 的工作不是“做 Agent”，而是把模型、消息、工具 schema、流式事件、usage、thinking、认证与多 Provider 差异抽象成统一 API。它定义 Context/Message/Model/Tool/AssistantMessageEvent 等协议，并把 OpenAI、Anthropic、Google 等适配器的原始协议转换为该统一表示。

- 可以单独用于模型网关或自定义聊天应用。
- 可以声明工具并接收 toolCall，但默认不会替你执行工具。
- 核心价值是把 Provider 差异压缩在 API adapter 边界。

3. pi-agent-core : 围绕模型构建状态机与 Agent Loop

pi-agent-core 增加 Agent 状态、消息 transcript、stream lifecycle、steering/follow-up 队列、工具执行与 before/after hook。其 Agent Loop 一直处理 AgentMessage，只有在真正请求 LLM 时才经 convertToLlm 转为 pi-ai Message。

因此它提供“会自动继续”的行为：assistant 返回 toolCall -> 查找工具 -> 校验参数 -> 执行 -> 构造 toolResult -> 再请求模型；这正是 pi-ai 自身不承担的职责。

4. pi-tui : 与 AI 无关的终端渲染框架

pi-tui 提供 TUI、Terminal、Editor、Text、Markdown、SelectList、Overlay、键盘协议、增量渲染和光标管理。它的 Component 接口本质上只是 render(width) 与 handleInput(data)；组件是否显示 LLM 消息完全由上层决定。

这使 pi-tui 可用于 Git 客户端、数据库终端等普通 TUI。它不应该 import pi-ai 或 pi-agent-core，否则 UI 基础设施会被领域语义污染。

5. pi-coding-agent : 产品编排与 Coding 领域层

pi-coding-agent 把底层能力组装成实际的 Pi：CLI、Interactive/Print/JSON/RPC、AgentSession、模型运行时、Session 树、Compaction、read/bash/edit/write 等工具、Extensions、Skills、Prompt Templates、项目资源与信任策略。

它的关键抽象 AgentSession 处在模式层与 Agent 之间：模式只处理 I/O，AgentSession 负责会话、模型、工具、资源、Compaction、扩展事件与持久化。

6. “能独立使用”要分两层理解

包	单独安装可用	依赖其他 Pi 包	不启动 TUI 可用	典型场景
pi-ai	是	无	是	统一模型访问、自建聊天/网关
pi-agent-core	是	pi-ai	是	自建 headless Agent / Web Agent
pi-tui	是	无	不适用	任意终端应用
pi-coding-agent	是	前三者	是	完整 Pi CLI 或高层 SDK

“pi-coding-agent SDK 可以 headless 使用”不等于“安装时没有 pi-tui”。运行路径可以不创建 InteractiveMode，但 npm 依赖仍包含 TUI。

7. 为什么这四层值得保持边界

设计边界	如果打破会怎样	当前收益
pi-ai 不依赖 Agent	Provider adapter 会知道业务循环	模型层可在任意应用复用
agent-core 不依赖 coding-agent	通用 Agent 被 read/edit/bash 等产品语义绑定死	Agent Loop 可独立测试/嵌入
pi-tui 不依赖 AI	终端组件变成 Agent 专用 UI	UI 框架可复用、易测试
coding-agent 负责组合	若底层互相横向依赖，替换和测试困难	产品层可以自由组合模型、Agent 与 UI

8. 一次请求跨包时的真实边界

```
InteractiveMode (coding-agent)
  -> AgentSession (coding-agent)
    -> Agent.prompt (agent-core)
      -> runAgentLoop (agent-core)
        -> streamFn(Model, Context)
          -> ModelRuntime / pi-ai Provider

response event
  -> pi-ai AssistantMessageEvent
  -> agent-core AgentEvent
  -> coding-agent AgentSessionEvent
  -> InteractiveMode
  -> pi-tui render
```

9. 修改某层时应避免的越界

- 增加新 Provider：优先落在 pi-ai；不要在 InteractiveMode 判断 provider 名称。
- 改变工具循环策略：优先落在 pi-agent-core；不要复制到每个运行模式。
- 增加终端通用控件：落在 pi-tui；具体 Agent 消息组件留在 coding-agent。
- 增加 Session/项目资源/Extension 产品能力：落在 coding-agent；不要塞进通用 Agent。

动态验证方案

实验 1：最小安装验证

假设：pi-ai 与 pi-tui 都可在不安装 coding-agent 的项目中工作

操作：创建三个空 npm 项目，分别只安装 pi-ai、pi-agent-core、pi-tui，运行 README 最小例子。

预期：pi-ai 可直接发模型请求；agent-core 自动带入 pi-ai 依赖；pi-tui 可显示纯文本编辑器。

若不一致：若打包失败，检查 workspace 包名、发布版本与 Node/Bun 条件。

实验 2：依赖越界检查

假设：底层包不应 import 上层包

操作：对 packages/ai、packages/agent、packages/tui 搜索 @earendil-works/pi-coding-agent。

预期：核心源码中不存在上层依赖。

若不一致：若出现，判断是测试/示例还是生产依赖。

源码证据索引

文件	关键符号/用途	本报告结论
packages/ai/package.json + src/index.ts	依赖与公共导出	pi-ai 为 Provider/LLM 基础层
packages/agent/package.json + src/agent.ts	只依赖 pi-ai；Agent 状态/循环	通用 Agent 层
packages/tui/package.json + src/tui.ts	无 Pi AI 依赖；Component/TUI	通用终端 UI 层

packages/coding-agent/package.json + src/core/agent-session.ts	同时组合三包	最终 Coding Agent 产品层
packages/coding-agent/src/core/sdk.ts	createAgentSession	证明 coding-agent 也可作为 SDK/headless 使用

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/ai/src/types.ts	先掌握统一消息/模型协议	是
2	packages/agent/src/agent.ts	看状态与 Agent facade	是
3	packages/agent/src/agent-loop.ts	看自动循环边界	是
4	packages/tui/src/tui.ts	理解 UI 与业务完全解耦	否
5	packages/coding-agent/src/core/agent-session.ts	看产品编排如何跨层	是